

Resumen de SOLID

Esteban Prieto Lugo

Ingesoft II – Sergio Andrés Vargas Pedraza

Los principios SOLID son 5 recomendaciones para dejar nuestros códigos limpios, ordenados y documentado. A continuación, los 5 principios:

- **Single Responsibility Principle (SRP):** Son módulos, clases o funciones que deben tener una sola razón para cambiar en el tiempo.
 - **Ejemplo:**

```
// Clase que solo representa un usuario
class Usuario {
    private String nombre;
    private String email;

    public Usuario(String nombre, String email) {
        this.nombre = nombre;
        this.email = email;
    }

    public String getNombre() {
        return nombre;
    }

    public String getEmail() {
        return email;
    }
}

// Clase que solo maneja la persistencia
class UsuarioRepository {
    public void guardar(Usuario usuario) {
        // Lógica para guardar en base de datos
        System.out.println("Guardando usuario en BD: " +
usuario.getNombre());
    }
}

// Clase que solo maneja emails
class EmailService {
```

```

public void enviarEmail(Usuario usuario, String mensaje) {
    // Lógica para enviar email
    System.out.println("Enviando email a: " + usuario.getEmail());
}
}

```

- **Ejemplo de incumplimiento del SRP:** Un ejemplo del incumplimiento es cuando las clases tienen muchas razones para cambiar, que gestione varias cosas, y para cambiar una lógica de negocios de cambiar un componente UI.
- **Open/Closed Principle (OCP):** El software debe estar abierto para añadir nuevo código y cerrado para no editar el código ya funcional.
 - **Ejemplo:**

```

// Interfaz base
interface Figura {
    double calcularArea();
}

class Circulo implements Figura {
    private double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}

class Rectangulo implements Figura {
    private double ancho;
    private double alto;

    public Rectangulo(double ancho, double alto) {
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override
    public double calcularArea() {
        return ancho * alto;
    }
}

```

```

class Triangulo implements Figura {
    private double base;
    private double altura;

    public Triangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    @Override
    public double calcularArea() {
        return (base * altura) / 2;
    }
}

// No necesitamos modificar esta clase para agregar nuevas figuras
class CalculadoraArea {
    public double calcularAreaTotal(List<Figura> figuras) {
        double total = 0;
        for (Figura figura : figuras) {
            total += figura.calcularArea();
        }
        return total;
    }
}

```

- **Ejemplo de incumplimiento del OCP:** Un ejemplo de incumplimiento que hay un nuevo tipo o flujo se toca un switch o un if.
- **Liskov Substitution Principle (LSP):** Las subclases deben poder sustituir a las clases base sin alterar el comportamiento esperado.
 - **Ejemplo:**

```

// Diseño correcto: Separamos las aves según su capacidad
abstract class Ave {
    public abstract String mover();
}

class AveVoladora extends Ave {
    @Override
    public String mover() {
        return "Volando por el cielo";
    }
}

class AveNoVoladora extends Ave {
    @Override

```

```

    public String mover() {
        return "Caminando en el suelo";
    }
}

class Pinguino extends AveNoVoladora {
    @Override
    public String mover() {
        return "Nadando en el agua";
    }
}

// Uso correcto
public class Main {
    public static void moverAve(Ave ave) {
        System.out.println(ave.mover());
    }

    public static void main(String[] args) {
        moverAve(new AveVoladora()); // Funciona correctamente
        moverAve(new Pinguino());    // Funciona correctamente
    }
}

```

- **Ejemplo de incumplimiento de LSP:** Errores de retorno, excepciones por falta de implementación de contrato, código roto tras sustituir la implementación.
- **Interface Segregation Principle (ISP):** Las piezas no deben obligarse a depender de interfaces que no usan.
 - **Ejemplo:**

```

// Interfaces pequeñas y específicas
interface Impresora {
    void imprimir(Documento documento);
}

interface Escáner {
    void escanear(Documento documento);
}

interface Fax {
    void enviarFax(Documento documento);
}

// Las clases implementan solo lo que necesitan
class ImpresoraSimple implements Impresora {
    @Override

```

```

    public void imprimir(Documento documento) {
        System.out.println("Imprimiendo documento");
    }
}

class Fotocopiadora implements Impresora, Escáner {
    @Override
    public void imprimir(Documento documento) {
        System.out.println("Imprimiendo");
    }

    @Override
    public void escanear(Documento documento) {
        System.out.println("Escaneando");
    }
}

class MultiFuncional implements Impresora, Escáner, Fax {
    @Override
    public void imprimir(Documento documento) {
        System.out.println("Imprimiendo");
    }

    @Override
    public void escanear(Documento documento) {
        System.out.println("Escaneando");
    }

    @Override
    public void enviarFax(Documento documento) {
        System.out.println("Enviando fax");
    }
}

```

- **Ejemplo de incumplimiento de ISP:** Interfaces gigantes que no usan todas sus propiedades, componentes con props no usados o indefinidos, funciones que reciben objetos cuyas propiedades no se usan.
- **Dependency Inversion Principle (DIP):** Depende de abstracciones, no de implementaciones concretas.
 - **Ejemplo:**

```

// Abstracción (interfaz)
interface Almacenador {
    void guardar(String datos);
}

```

```

// Implementaciones de bajo nivel
class BaseDeDatos implements Almacenador {
    @Override
    public void guardar(String datos) {
        System.out.println("Guardando en MySQL: " + datos);
    }
}

class Archivo implements Almacenador {
    @Override
    public void guardar(String datos) {
        System.out.println("Guardando en archivo: " + datos);
    }
}

class CloudStorage implements Almacenador {
    @Override
    public void guardar(String datos) {
        System.out.println("Guardando en AWS S3: " + datos);
    }
}

// Módulo de alto nivel depende de abstracción
class Servicio {
    private Almacenador almacenador;

    // Inyección de dependencia
    public Servicio(Almacenador almacenador) {
        this.almacenador = almacenador;
    }

    public void procesar(String datos) {
        // Procesa datos
        System.out.println("Procesando datos...");
        almacenador.guardar(datos);
    }
}

// Uso
public class Main {
    public static void main(String[] args) {
        Servicio servicio1 = new Servicio(new BaseDeDatos());
        Servicio servicio2 = new Servicio(new Archivo());
        Servicio servicio3 = new Servicio(new CloudStorage());
    }
}

```

```
servicio1.procesar("Información importante");  
}  
}
```

- **Ejemplo de incumplimiento de DIP:** Necesito reconfigurar el módulo para probar, importas y creas dependencias en lugar de inyectarlas y recibirlas.